iPRA with one explicit computation schema

Canonical successor-seeded syntax and complete-checker accounting

Prepared for Vladimir Reshetnikov

12 November 2026

Abstract

We give a presentation of intuitionistic primitive recursive arithmetic with six inference patterns, two numerical constructors in addition to variables, and one explicit computation schema. Successor is defined by a successor-seeded recursor. Its zero computation is imposed as a canonical spelling of terms, and normalized substitution is specified and charged. We prove both translations to the existing arithmetic and both composites, including the recursor body-extensionality lemma required by normalization. A complete strict checker has 4,611 normalized bytes against 4,923 for a matching seven-rule scalar control, a 6.34 percent reduction. A separate 516-byte refinement improves the existing compact guarded calculus and is not assigned to the new axiomatization. We also prove two boundaries: a scalar self-seeded language cannot define predecessor, and induction on zero equations does not recover arbitrary equation induction, even with coverage and successor injectivity. The latter uses a nonstandard two-point-fiber countermodel. All new metatheorems are written proofs; the archive includes executable tests, exact source controls, and reproduction instructions, without claiming new proof-assistant verification or an absolute minimum.

Contents

1 Result and comparison contract 3

2 Scope and the exact mathematical change 3

3 Canonical terms and substitution 4

4 The complete six-rule calculus 5

5 First establish the unrestricted comparison 6

6 Only then reduce the canonical calculus to six rules 8

7 Arithmetic bridge and both interpretation composites 9

8 What has and has not become simpler 10

9 Complete implementations and measured complexity 11

10 Why deleting the initializer is a different problem 13

11 Zero-equation induction does not recover equation induction 15

12 Further research questions 20

A Native bootstrap: cases before a full selector 21

B A successor-seeded scalar basis with two computation schemas 24

C Complementary introductions with one local assumption 30

D Arithmetic strength and both ordinary interpretation composites 36

E Internal scalar predecessor bootstrap 37

F From literal-equation induction to full intuitionistic PRA 41

G Pairing, primitive recursion, and internal numerical composites 47

H The complete strict canonical checker 49

I The separate compact scalar byte profile 51

1 Result and comparison contract

The proposed presentation has six primitive inference patterns and the numerical term constructors 0 and R, in addition to variables. Successor is a derived notation. There is one explicit computation schema. Its ordinary realization is bi-interpretable with full first-order intuitionistic primitive recursive arithmetic, with the entire numerical domain in both directions and with identity graph $x = y$ for both numerical composites.

The reduction is precise but has a cost: the zero computation becomes a canonical-syntax convention, and substitution must enforce that convention. We count this normalization in the complete implementation. A reduction from seven named proof rules to six is therefore not, on its own, a proof of a smaller complete checker.

Feature	Current scalar presentation	Canonical R presentation
Numerical constructors	$0, S, I$	$0, R$
Explicit computation schemas	Two	One
Primitive inference patterns	Seven	Six
Term equality in the checker	Literal syntax	Canonical syntax
Substitution	Capture avoiding	Capture avoiding, then normalize
Numerical interpretation domain	Whole	Whole
Numerical composite graph	$x = y$	$x = y$

The comparison is pinned to ProveIt commit `8eb71c1e570e07ef45ead686628e77f18ef34798`, inspected on 12 November 2026, UTC. The current seven-rule scalar calculus is `MPrf` in `NativeMinimal.lean`. The separate crossed calculus `RPrf` is in `NativeReversal.lean`. Their equivalences with the native scalar core are already formalized upstream. The present canonical R reduction has a written proof and executable implementation tests; it is not a new Lean or Rocq formalization.

The repository's smallest compact profiles are a different comparison category. The selected byte profile has 522 normalized source bytes, but its bi-interpretation theorem concerns a guarded public language, with broader auxiliary proof syntax. Its compact program does not itself validate that public grammar. Our strict canonical-R checker validates its full public language. We therefore report two separate comparisons: matching strict implementations of the mathematical calculi, and small implementation changes to the existing compact calculus. No compact size is attributed to the new six-rule mathematics.

The successor-seeded recursor, its predecessor construction, the scalar arithmetic bridge, crossed transport, and witness-directed introduction are inherited. The mathematical change developed here is canonicalization of the successor-seeded syntax and the proof that it preserves substitution, derivability, and both interpretation composites. A separate strengthening of the scalar self-seeded obstruction is proved in Section 10.

2 Scope and the exact mathematical change

The comparison source is ProveIt commit `8eb71c1e570e07ef45ead686628e77f18ef34798`. The successor-seeded recursor and its arithmetic completeness are inherited from `Research/successor_seed_completion/` and `Research/scalar_successor/`. The crossed-transport argument is the retraction criterion of `Research/reversal_core/theory/reversal_equivalence.tex`. The single-introduction argument is the idea formalized in `Lean/ConciseIPRA/NativeOneIntro.lean`. The new construction here is their application to explicitly canonical successor-seeded syntax, with a proof of the resulting change of term equality and substitution.

No independent successor constructor is used. Raw terms are

$$t ::= x \mid 0 \mid R(n, a; u.h).$$

Only u in h is bound. Put

$$Sa := R(0, a; u.0), \qquad 1 := S0.$$

Bound variables are renamed to avoid capture. There are no numerical variables over function descriptions: a body is finite earlier syntax, with its remaining free variables as passive parameters.

3 Canonical terms and substitution

Define a normalization of raw terms recursively:

$$\nu(x) = x, \qquad \nu(0) = 0,$$

$$\nu(R(n, a; u.h)) = \begin{cases} R(0, \nu(a); u.0), & n \equiv 0, \\ R(\nu(n), \nu(a); u.\nu(h)), & n \not\equiv 0. \end{cases}$$

Here $n \equiv 0$ means literal syntactic identity with zero. It is not a numerical equality test. A canonical term is a fixed point of ν. Equivalently, every subterm with literal zero counter has literal zero body. This is a decidable grammar of finite expressions.

The normalizer preserves the outer term constructor:

$$\nu(t) \equiv 0 \quad \Longleftrightarrow \quad t \equiv 0.$$

In particular $\nu(Sa) = S\nu(a)$. It is idempotent, respects renaming of bound and free variables, and introduces no new free variables.

Extend ν to native formulas and contexts by normalizing all terms:

$$A ::= (s = t) \mid Q_y A \mid D_{x,y} A.$$

The roots $=$, Q, and D are preserved. All terms in a canonical formula are canonical.

Lemma 3.1 (Substitution coherence)**.** *For every raw term or formula T and capture-free simultaneous numerical substitution σ,*

$$\nu(T[\sigma]) = \nu(\, \nu(T)[\nu \circ \sigma]\,).$$

Proof. Use structural induction after freshening bound names. The only additional case is an R-node. If its original counter is literally zero, both sides erase the body. If it is not literally zero but becomes zero under substitution, both sides again erase the body. In the remaining case both sides retain the R-node and the equality follows from the induction hypotheses for counter, initializer, and body. The fact that normalization never turns a nonzero-root constructor into literal zero justifies the case division. Formula binders introduce no extra case. □

For canonical expressions define

$$T\langle\sigma\rangle := \nu(T[\sigma]).$$

This is the substitution operation used by every rule below. Lemma 3.1 gives associativity of normalized substitution and its compatibility with renaming. A variable substituted by zero may expose a zero-counter node. Normalizing only the original syntax, or testing the unsubstituted counter, is insufficient.

In a canonical-calculus display, an expression $R(n, a; u.h)$ denotes the normalization of that raw constructor expression. This smart-constructor convention is relevant when $n \equiv 0$: its written body is discarded. In the raw comparison calculus below, constructor expressions instead retain their literal bodies. Substitution may accept a raw image because it normalizes the completed result; equivalently it may first normalize that image by Lemma 3.1.

The sole primitive computation schema is

$$R(Sn, a; u.h) = h\langle \nu(R(n, a; u.h))/u \rangle. \tag{R_+}$$

All displayed terms are canonical: the left counter Sn is never literally zero. The raw zero computation $R(0, a; u.h) = Sa$ is now definitional, because both sides have the same canonical spelling.

4 The complete six-rule calculus

Contexts are finite reusable lists of canonical native formulas. Define root reversal by

$$(s = t)^\dagger = (t = s), \qquad (Q_y B)^\dagger = Q_y B, \qquad (D_{x,y} B)^\dagger = D_{x,y} B.$$

It commutes with canonical substitution.

The calculus $\mathcal{K}_\nu$ has the following six inference patterns.

1. **Hypothesis:** $\Gamma \vdash A$ when $A \in \Gamma$.

2. **Successor computation:** every instance of (R_+), in every context.

3. **Crossed equational transport:**

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash E\langle s/v \rangle}{\Gamma \vdash (E\langle t/v \rangle)^\dagger}, \qquad s \not\equiv 0. \tag{X}$$

   Here $E(v)$ is a literal-equation template. Constant templates are allowed.

4. **Crossed closure elimination:**

$$\frac{\Gamma \vdash Q_y B(y) \qquad \Gamma \vdash B\langle 0/y \rangle}{\Gamma \vdash (B\langle t/y \rangle)^\dagger}. \tag{QX}$$

5. **Paired elimination:**

$$\frac{\Gamma \vdash D_{x,y} B(x, y) \qquad \Gamma, Q_y B\langle b/x \rangle \vdash A}{\Gamma \vdash A}. \tag{DE}$$

   The eigenparameter b is fresh for the retained context, conclusion, and fixed parameters of B.

6. **One introduction:**

$$\frac{\Gamma, B\langle t/x, \eta/y \rangle \vdash B\langle t/x, Sz/y \rangle}{\Gamma \vdash D_{x,y} B}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{WI}$$

   Here z is fresh for the retained context, witness t, and fixed parameters of B.

There is no primitive reflexivity, symmetry, numerical case rule, induction rule, zero-computation rule, ordinary logical connective, or general numerical-substitution rule. The designated contradiction is $0 = S0$.

The intended ordinary readings are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y),$$

$$(D_{x,y} B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big).$$

All numerical terms use the successor-seeded primitive-recursive interpretation of R. Domains are whole, and equality is actual equality.

5 First establish the unrestricted comparison

Do not initially attempt a substitution theorem for the six-rule calculus. Literal-zero source and witness tests are not stable under arbitrary numerical substitution. Instead use two unrestricted comparison calculi.

Definition 5.1 (Raw and canonical unrestricted calculi)**.** The raw calculus $\mathcal{R}_{\text{free}}$ uses raw R-terms, ordinary capture-free substitution, the two computation schemas

$$R(0, a; u.h) = Sa, \qquad R(Sn, a; u.h) = h[R(n, a; u.h)/u],$$

Hyp, unrestricted transport in every native formula template, ordinary QE, DE, and the two introductions

$$\frac{\Gamma, B(t,0) \vdash B(t, Sz)}{\Gamma \vdash D_{x,y} B}, \qquad \frac{\Gamma, E(t,z) \vdash E(t, Sz)}{\Gamma \vdash D_{x,y} E} \quad (E \text{ a literal equation}).$$

The canonical calculus $\mathcal{C}_\nu$ has the same unrestricted rules using canonical substitution, but has only R_+ as its computation schema.

Here ordinary QE is QX without root reversal. Unrestricted transport is X without either its literal-zero source restriction or its root reversal, and it permits arbitrary native formula templates. The freshness conditions for DE and both introductions are the conditions already stated for DE and WI. Hypothesis and DE are unchanged.

Lemma 5.2 (Structural rules)**.** *Weakening, eigenvariable renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{R}_{\text{free}}$ and $\mathcal{C}_\nu$. Weakening, eigenvariable renaming, and ordinary-assumption cut are admissible in $\mathcal{K}_\nu$, without presupposing numerical substitution there.*

Proof. Induct on derivations, freshening local eigenparameters first. For $\mathcal{C}_\nu$, coherence of normalized substitution proves closure of the computation schema and all displayed instances. The case/progress distinction in the unrestricted calculi depends only on the formula root and is stable. Cut replaces assumption uses with proofs without changing witnesses, roots, or source terms; hence it remains valid in the restricted six-rule calculus. Renaming preserves literal zero and root constructors. This proof makes no assertion that general numerical substitution directly preserves an X or WI node. □

Lemma 5.3 (Elementary resources)**.** *Both unrestricted calculi derive reflexivity, symmetry, transitivity, numerical congruence, ordinary Q-introduction by cases, numerical cases, and contextual induction on literal equations.*

Proof. Define the retraction context

$$J(a) := R(S0, 0; u.a).$$

The successor computation gives $J(a) = a$, even in $\mathcal{C}_\nu$. Use this equation twice in transport with motive $v = a$ to get $a = a$. The usual equality motives give symmetry, transitivity, and numerical congruence.

From $\Gamma, B(0) \vdash B(Sz)$, case introduction with an unused witness gives $D_{w,y}B(y)$; DE followed by Hyp exports $Q_yB(y)$. Applying QE with supplied base and successor proofs gives numerical cases. Repeating the unused-witness construction with progress introduction gives equation induction from its ordinary base and step premises. No selector, predecessor, or ordinary logical connective is needed. □

Lemma 5.4 (Recursor body extensionality)**.** *Suppose, for a fresh numerical variable v, that a fixed context proves $h(v) = k(v)$. Then it proves*

$$R(n, a; u.h(u)) = R(n, a; u.k(u)).$$

The assertion holds in both unrestricted calculi.

Proof. Apply contextual literal-equation induction to the equation comparing the two trajectories at a fresh counter. At zero both terms are Sa, by zero computation in $\mathcal{R}_{\text{free}}$, and by canonical syntax and reflexivity in $\mathcal{C}_\nu$. At the successor, computation exposes the two updates. Use the induction equation to replace the old state in h, then use the supplied pointwise body equality at the second old state. Such instantiation is justified by Lemma 5.2, which has already been proved for these unrestricted calculi. All passive parameters and the initializer remain fixed during the induction. □

This lemma is necessary. Changing a normalized body changes its finite recursor description; it is not generally ordinary congruence of one fixed first-order function symbol.

Lemma 5.5 (Normalization is provable in the raw calculus)**.** *For every raw term t,*

$$\mathcal{R}_{\text{free}} \vdash t = \nu(t).$$

For every raw native formula A, the raw calculus derives both $A \vdash \nu(A)$ and $\nu(A) \vdash A$, uniformly in its free parameters.

Proof. For terms use structural induction. Counter and initializer changes use numerical congruence. The body change uses Lemma 5.4 and the induction hypothesis with the body's bound variable fresh and free in the auxiliary proof. If the normalized counter is zero, use the raw zero computation to replace the body by zero; otherwise the resulting term is already its normal form.

For formulas use external induction on the native formula syntax. The equation case follows from the two endpoint identities. For the Q case, under $Q_yA(y)$, introduce $Q_y\nu(A(y))$ by cases: convert its assumed normalized base to the old base, use QE at the fresh successor, and convert back. The reverse direction is identical. For D, eliminate the old D at a fresh witness, introduce the new D at that witness by cases, convert the new base to the old base, use the exported old Q, and convert the successor conclusion. Cut closes the construction. These are explicit derived binder-congruence arguments, not applications of an assumed ordinary quantifier logic. □

Theorem 5.6 (Canonical and raw consequence)**.** *For canonical Γ, A,*

$$\Gamma \vdash_{\mathcal{C}_\nu} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{R}_{\text{free}}} A.$$

More generally, normalization sends every raw derivation to a canonical derivation of its normalized sequent.

Proof. For the raw-to-canonical direction normalize the entire finite derivation. Coherence of substitution handles every instance of transport, QE, DE, and either introduction. A raw zero-computation leaf normalizes to $Sa = Sa$, which is derived by Lemma 5.3; a successor-computation leaf normalizes to an R_+ instance.

For the converse, regard canonical expressions as raw expressions and recurse on the canonical derivation. Any canonical rule instance differs from its raw counterpart only by normalizations of substituted expressions. Lemma 5.5 converts these premises, local assumptions, and conclusions in both directions. Use ordinary-assumption cut for local assumptions. For example, canonical transport first converts the canonical minor $\nu(E[s/v])$ to raw $E[s/v]$, applies raw transport, and converts $E[t/v]$ to its canonical normalization. Introductions and DE use the same conversions under their retained local assumptions. Freshness is preserved because normalization introduces no free variables. No restricted-rule substitution theorem enters either direction. □

6 Only then reduce the canonical calculus to six rules

6.1 Equality before introduction repair

In $\mathcal{K}_\nu$, keep $J(a) = R(S0, 0; u.a)$ and $c = J(0)$. Each $J(a)$ is literally nonzero and $J(a) = a$ is an R_+ instance.

X with major $c = 0$ and a constant equation template reverses any proved equation. Compound root reversal is the identity. Therefore QX followed by reversal supplies ordinary QE, and X followed by reversal supplies ordinary equational transport at nonzero literal sources.

Reflexivity follows by using $J(a) = a$ as both X premises with template $v = a$. To remove the literal-zero source restriction, use the established retraction argument, whose short details are included here.

If $p, q \not\equiv 0$, $p = 0$, and $q = 0$, then X with major $p = 0$, minor $J(p) = p$, and template $J(v) = p$ yields $p = c$. Similarly $q = c$. Reverse and join through the allowed source c to obtain $p = q$. Transitivity now holds without restriction: if its middle term is nonzero use restricted ordinary transport; if it is zero and both endpoints are nonzero, reverse the second equation and use the zero join; literal-zero endpoint cases are immediate.

Numerical congruence along a nonzero source is ordinary restricted transport from reflexivity. For an equation $0 = t$ with $t \not\equiv 0$, reverse it to $t = 0$; transport the reflexivity $f(t) = f(t)$ in the template $f(v) = f(t)$, giving $f(0) = f(t)$. If both endpoints are zero, use reflexivity. Finally, from $s = t$ and $a(s) = b(s)$, numerical congruence and unrestricted transitivity give

$$a(t) = a(s) = b(s) = b(t).$$

Thus unrestricted ordinary transport in literal-equation templates is available before any introduction repair. All steps instantiate explicit schemas at the terms at hand; none substitutes into a restricted derivation.

6.2 Recovering both introductions

Lemma 6.1 (Case introduction)**.** *Every unrestricted case-introduction instance is derivable in $\mathcal{K}_\nu$.*

Proof. WI already is case introduction when its matrix is compound or its witness is not literally zero. Only an equation matrix E at witness zero remains.

Use instead the witness $c = J(0)$, which is nonliteral and provably zero. Under the local assumption $E(c, 0)$, unrestricted equation transport gives $E(0, 0)$. Cut in the given case-premise

derivation to get $E(0, Sz)$; transport back to $E(c, Sz)$. WI at witness c is the case choice and concludes the required unchanged formula $D_{x,y}E(x, y)$. The new witness contains no eigenparameter z. □

Lemma 6.2 (Progress introduction)**.** *Every unrestricted literal-equation progress instance is derivable in $\mathcal{K}_\nu$.*

Proof. Suppose the supplied premise is $\Gamma, E(t, z) \vdash E(t, Sz)$. Let the first coordinate w be unused in the equation $E(t, y)$. WI with witness zero proves

$$\Gamma \vdash D_{w,y}E(t, y)$$

by its progress branch. Under the additional assumption $E(t, 0)$, eliminate this paired formula. Its exported clause is $Q_yE(t, y)$, independent of the fresh witness. Ordinary QE gives $E(t, Sz)$. Closing DE gives the premise required for the recovered case introduction at the original witness t, which concludes $D_{x,y}E(x, y)$. □

6.3 Compound transport

Now equation transport, ordinary QE, DE, and both introductions are available. Derive ordinary transport in all native formula templates by external induction on their constructor structure. The equality case is already proved. For a Q-template, use derived case Q-introduction: convert its new base back along the equality by the induction hypothesis, apply the old Q at the fresh successor, and convert forward. For a D-template, DE chooses the old witness; case-introduce the new D at that witness, convert its base back, use the exported old clause, and convert the successor forward. Reverse the numerical equality to obtain both directions when required. The derivations use only contextual equation symmetry, cut, and renaming already established above.

Theorem 6.3 (Six-rule equivalence)**.** *For every canonical native sequent,*

$$\Gamma \vdash_{\mathcal{K}_\nu} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{C}_\nu} A.$$

Proof. The preceding constructions provide every rule of $\mathcal{C}_\nu$ inside $\mathcal{K}_\nu$. Conversely, a WI instance is exactly one of the two unrestricted introductions, and the crossed rules are ordinary transport or QE followed by derived equality symmetry when their conclusion is an equation. Hyp, R_+, and DE are common. Transform finite derivations recursively. No numerical-substitution admissibility for $\mathcal{K}_\nu$ was presupposed; it may now be transferred through the equivalence. □

Syntactic tests under normalization. Normalization preserves literal-zero status of a whole term and the root constructor of a formula, so it does not itself alter an X or WI test. Arbitrary numerical substitution can alter such tests. Keeping the unrestricted equivalence and the later restricted-rule repair separate avoids confusing these two facts.

7 Arithmetic bridge and both interpretation composites

The full arithmetic bootstrap for $\mathcal{R}_{\text{free}}$ is inherited, not a new claim of this note. Appendices A and B reproduce the pinned native finite-case repair and successor-seeded predecessor construction. Their recursor is denoted J rather than R; renaming that one constructor makes it the present raw language.

Their explicit term P is proved, with only native cases, equality reasoning, and literal-equation induction, to satisfy

$$P0 = 0, \qquad P(Sa) = a.$$

No predecessor, successor cancellation, numerical equality test, ordinary selector, or compound-formula induction is assumed during that bootstrap. The native finite-case appendix is required: theorem-level numerical cases alone would not justify arbitrary contextual finite case splits before the logical macros exist.

The initialized scalar iterator is then the finite raw R-term

$$\widehat{I}(n, a; u.h) = P\big(R(n, a; v.S\big(h[P(v)/u]\big))\big) .$$

Its zero computation is $P(Sa) = a$. At a successor, R_+ exposes an outer successor that P removes, leaving exactly

$$\widehat{I}(Sn, a; u.h) = h[\widehat{I}(n, a; u.h)/u].$$

The unrestricted selector $C(n, a, b) := \widehat{I}(n, a; u.b)$ is now available. The established native scalar bridge reconstructs all intuitionistic first-order connectives, actual-equality reflection, quantifier-free induction, pairing, and every fixed primitive-recursive definition. That bridge is separately formalized in the pinned repository; its validity is inherited here.

Conversely, interpret a raw successor-seeded description by

$$R(n, a; u.h) \longmapsto I(n, Sa; u.h).$$

The two raw computations are instances of the scalar computations. For the two numerical interpretation composites, first identify successor: its definition $R(0, a; u.0)$ translates to $I(0, Sa; u.0) = Sa$. Then proceed through each fixed description in dependency order. After earlier descriptions have been identified, the description and its double translation have the same initial value and the same update. Literal-equation induction on the equation comparing their trajectories proves equality throughout the entire numerical domain. The initializer and passive parameters remain independent and fixed during this induction, and are instantiated only afterward. Body equality is used through the derived trajectory comparison, not as an uncharged recursor-congruence axiom.

The native formula round trips use the established logical macros once C is available. The Q and D readings are those already displayed; their inverse translations are interderivable by case introduction, DE, and QE. Formula congruence and the numerical identities extend the round trips to every formula.

Combining these inherited arithmetic and logical results with Theorems 5.6 and 6.3 gives the precise conclusion:

$\mathcal{K}_\nu$ is equiconsistent and bi-interpretable with full first-order iPRA.

Both numerical domains are whole and interpreted equality is actual equality. The numerical composite isomorphisms have graph $x = y$. On raw syntax the composite through canonical syntax is provably, rather than literally, the identity; on canonical syntax normalization is literally idempotent.

8 What has and has not become simpler

There are six explicit inference patterns instead of seven, and no independent successor constructor. The omitted zero computation has become a canonical-syntax law:

$$R(0, a; u.h) \equiv R(0, a; u.0).$$

That law must be enforced by every substitution and comparison used by a complete checker. It has not disappeared from the trusted specification.

For the current compact encodings, a numerical recursor and a Q-formula both have arity three, so their opener must distinguish numerical depth before applying this normalization. Testing the original counter instead of the substituted one is unsound. Merely rejecting noncanonical input also fails: internally generated substitution instances must be normalized.

Accordingly this theorem establishes a mathematical change of presentation and a smaller explicit rule inventory. The separately measured matching strict implementation is also smaller; no absolute or compact-checker minimum for this new calculus follows. A fair measurement must include canonicalization, syntax and scope handling, decoding, every inference rule, and the public input/output driver. Any separate implementation improvement for the old scalar calculus must not be attributed to this new canonical R calculus.

9 Complete implementations and measured complexity

9.1 Matching strict implementations

The directory `strict/` contains three standalone Python modules. Each imports only `json` and `sys`; none imports a proof producer, axiom database, normalization oracle, or test utility. The public input is a JSON object with exactly `goal` and `proof` fields. The goal must be closed and satisfy the entire stated grammar. A successful run exits with status zero and prints exactly `ACCEPT`; every failed check rejects.

The three modules use the same proof-tree format, the same context handling, the same de Bruijn convention, the same logical rules, and the same public driver. The scalar control has explicit successor and initialized iteration. It uses crossed eliminations and witness-directed introduction, so it is an equivalent matching presentation of the scalar theory, rather than a literal transcription of the uncrossed `MPrf` declaration. The raw-R control replaces successor by its recursor abbreviation and keeps both computations. The canonical-R module additionally normalizes every recursor construction, validates canonical inputs, and deletes the explicit zero-computation rule. This is a comparison of complete matching implementations, not a comparison of a fragment against a complete module.

Complete module	Tokens	AST	Bytes	Gzip 9
Scalar I, seven rules	1433	1595	4923	1224
Raw R, seven rules	1378	1532	4753	1209
Canonical R, six rules	1338	1485	4611	1205

Matching strict implementations; normalization and both interfaces are charged.

The canonical program saves 312 normalized bytes against the scalar control, or 6.34%, together with 95 significant tokens and 110 AST nodes. Against the raw-R control it saves 142 normalized bytes. Thus normalization has been charged, and the resulting strict implementation is still smaller in this matched design. These programs are intentionally transparent; their size does not improve the repository's much more compressed arity codec.

Measurements use CPython 3.12.14. Actual docstrings are removed, the complete module is normalized with `ast.unparse`, and one final LF is appended. Significant tokens exclude comments, indentation, dedentation, and line-end and end-marker tokens. AST nodes are counted with `ast.walk`. The last column is gzip level 9 of the normalized module with timestamp zero; it is an

informational coordinate, not the optimization objective. Raw source hashes and sizes are recorded separately.

9.2 Checking syntax, normalization, and binders

In the strict format, 0 is JSON integer zero, positive integers are de Bruijn variable indices, and $[n, a, h]$ is an R term whose body binds index one. Formula tags are `=`, `Q`, and `D`. In a D matrix, the universal variable has index one and the existential variable index two. Booleans, floats, negative indices, unknown tags, and extra fields are rejected where the specification requires an integer, formula, or fixed rule record.

The smart constructor is the complete normalization operation at a newly constructed recursor node:

$$\mathrm{rec}(n, a, h) = [n, a,\ 0 \text{ if } n = 0 \text{ else } h].$$

Its children have already been normalized by recursive substitution. Counter and initializer are traversed at the current binding depth; the body is traversed one binder deeper. Substituted images are shifted under binders. The implementation therefore normalizes a zero counter newly exposed by substitution, and it never mistakes a bound state for a free parameter.

The strict test suite contains 86 named certificates across the three calculi. All 172 command-line replays of the readable and normalized modules match their expectations. The canonical calculus has 12 positive and 18 negative certificates. Its positive cases include each inference mechanism, an actually used induction hypothesis, a paired elimination with a genuine witness and shifted retained context, and zero-counter normalization exposed by transport. Negative controls include a future witness, an escaping eigenparameter, an invalid source, a wrong closure base, a compound transport template, malformed syntax, and use of the removed zero rule.

There are also 480 independent numerical evaluations comparing substitution with environment evaluation, plus direct tests of nesting and capture. These are finite executable controls. The universal soundness and completeness argument is the correspondence between the six checked rule cases and the calculus proved equivalent above; the numerical tests alone do not establish that metatheorem.

9.3 Separate compact implementation refinement

The archive also contains a small refinement of the *existing* guarded scalar calculus. It has no role in the canonical-R equivalence theorem. The source transformations pair the already checked major and target comparisons, use a shorter integer/list discriminator, and compose the existing scalar-zero administrative truth convention with the guarded checker. A conservative variant changes only the boolean spelling of the expected major and preserves all JSON executions of its parent.

Complete module	Tokens	AST	Bytes	Gzip 9
Frozen compact byte control	265	303	522	329
Refined compact byte profile	265	306	516	332
Frozen compact AST control	262	281	606	348
Refined compact AST profile	263	280	602	348
Matched strict guarded control	541	620	1381	633
Refined strict guarded checker	542	624	1376	633

Existing scalar calculus only. Byte and AST improvements use different modules.

The compact byte and AST coordinates belong to different programs. Their public-language qualification remains the inherited guarded Eq/D contract: the compact programs do not themselves

validate the public grammar. The directory includes a separately counted strict goal and integer/list certificate front end, frozen controls, converted fixtures, an exact description of the changed neutral marker, and a reproducible measurement and test script. The compact gain is an implementation result, not a new arithmetic basis and not the size of the canonical-R checker.

The eager paired comparison may inspect previously unused instruction fields. Its completeness statement is therefore made for the specified well-scoped integer/list certificate language, and no equality of arbitrary malformed-JSON behavior is claimed for that variant. Full statements and the execution evidence are in the compact directory. No failed experimental checker is supplied as an accepted implementation.

10 Why deleting the initializer is a different problem

One might try to remove the initializer argument altogether. This section gives a new scalar-specific obstruction to that route. The argument is about standard term functions, not about a lower bound for all possible axiomatizations or proof checkers.

Consider terms generated by variables, 0, S, and the scalar constructor

$$F(0; v.h) = S(h(0)), \qquad F(Sn; v.h) = h(F(n; v.h)).$$

Arbitrary nesting and passive numerical parameters are allowed. There is no primitive numerical equality characteristic or additional arithmetic operation. The successor may instead be the abbreviation $F(0; v.a)$.

Theorem 10.1 (A restriction on every unary scalar term)**.** *For every unary term $t(n)$ in this language, there is a natural number C such that*

$$\forall n \in \mathbb{N} \quad \big(t(n) \le C \ \vee \ t(n) \ge n\big).$$

Consequently no such term defines predecessor $\max(n-1, 0)$*, integer halving, or any function having unbounded values strictly below its input.*

The counter-aware tied-seed language in the existing `local_scopes` package already has a finite-control obstruction, but it does define predecessor. Thus the unary conclusion above is specific to removing access to the old counter as well as the independent initializer. We strengthen the existing finite-control invariant; its original complete proof is included in the archive as `reference/finite_profiles.tex`.

Definition 10.2. Let $\mathcal{U}$ be the unary functions $f : \mathbb{N} \to \mathbb{N}$ for which there is C satisfying $f(n) \le C$ or $f(n) \ge n$ at every input.

Lemma 10.3 (Closure properties)**.** *Constants, the identity, and successor belong to $\mathcal{U}$. The class is closed under composition and under pointwise selection from a fixed finite family of its members.*

Proof. For finite selection, take the maximum of the finitely many bounds. For composition $f \circ g$, fix bounds C_f, C_g. If $g(n) \le C_g$, then the output is bounded by $\max_{j \le C_g} f(j)$. Otherwise $g(n) \ge n$, and the output is either at most C_f or at least $g(n) \ge n$. □

Lemma 10.4 (The orbit lemma)**.** *If $h \in \mathcal{U}$, then the sequence*

$$r(0) = 1 + h(0), \qquad r(n+1) = h(r(n))$$

viewed as a unary function of n, also belongs to $\mathcal{U}$.

Proof. A finite orbit is bounded. An infinite orbit has no repeated value, since a repeat in a deterministic orbit makes its future periodic. Fix a bound C for h. A strictly decreasing transition must end in $\{0, \dots, C\}$, so an injective orbit has only finitely many such transitions. After the last one, the orbit increases strictly. Eventually it exceeds every value in the earlier finite prefix. For all sufficiently large n, its first $n+1$ distinct values therefore lie in $[0, r(n)]$, and $r(n) \geq n$. Increase a constant to cover the finitely many earlier outputs. □

Proof of Theorem 10.1. Strengthen the finite-control invariant as follows. Every positive-arity term has one active numerical coordinate; each other coordinate is observed through a finite unary feature map; once those finite observations are fixed, the output is a unary profile of the active coordinate. Require every such profile to belong to $\mathcal{U}$. Constants and projections satisfy this property. In a composition, fixing the inactive features leaves a pointwise finite selection of compositions of old profiles. The closure lemma applies.

For F, first treat its counter as an independent input. If the active coordinate of its body is the state v, fixing the finitely many passive features gives finitely many updates $h_c \in \mathcal{U}$. Their orbit profiles belong to $\mathcal{U}$ by the orbit lemma, and the counter is the new active coordinate.

If instead the active coordinate of the body is a passive parameter p, write

$$h(v, p, \vec{q}) = H_{\beta(v),\gamma(\vec{q})}(p),$$

where β, γ take finitely many values and every $H_{b,c}$ belongs to $\mathcal{U}$. Form a finite catalogue $V_j(p)$ of all update profiles $H_{b,c}(p)$ and all initial profiles $1 + H_{\beta(0),c}(p)$. Every orbit state is a catalogue value, and every catalogue member belongs to $\mathcal{U}$.

The finite feature

$$\kappa(p) = (\beta(V_j(p)))_j$$

determines every transition of the catalogue indices once $c = \gamma(\vec{q})$ is fixed. For every possible c, k, let $\theta_{c,k}(n)$ be that finite-state process after n updates. Pack all these finitely many processes into the finite unary feature $\eta(n) = (\theta_{c,k}(n))_{c,k}$. Then

$$F(n; v.h) = V_{\eta(n)_{\gamma(\vec{q}),\kappa(p)}}(p).$$

For fixed features of $n, \vec{q}$, this is a pointwise finite selection of the catalogue profiles of p, so it belongs to $\mathcal{U}$. This argument uses only the displayed finite observations, not a numerical equality oracle. Substitute the actual counter term by the already established composition closure. Structural induction is complete. At arity one there are no inactive coordinates, yielding the asserted unary conclusion. □

For predecessor, choose $n > C+1$; its value is then greater than C and less than n, a contradiction. Halving fails at arbitrarily large $n > 2C + 1$. The theorem by itself concerns term definitions. It must not be presented as an obstruction to arbitrary graph interpretations or changed domains. Those broader questions are treated separately in the existing relational obstruction, also supplied in the archive.

10.1 Independence of the original two scalar computations

Neither computation equation of the original initialized scalar iterator can simply be deleted, even if full induction is supplied. In the standard natural numbers, interpret every fixed body compositionally.

If I_0 is omitted, put $I(n, a; h) = h^n(0)$. The successor equation holds, but $I(0, 1; u.u) = 0 \neq 1$. If I_S is omitted, put $I(n, a; h) = a$. The zero equation holds, but the successor equation fails for

$h(u) = Su$. Both structures have ordinary numerical coverage, separation, and every first-order induction instance. Thus the canonical-R construction succeeds by changing the representation; it does not derive a deleted scalar law from the unchanged remaining axioms.

11 Zero-equation induction does not recover equation induction

Consider the initialized scalar language

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h)$$

with its two computation schemas

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

Only the state variable u is bound in h; the other variables of the fixed finite body are numerical parameters. Use ordinary intuitionistic first-order logic with actual equality, numerical coverage

$$\forall x\,(x = 0 \vee \exists y\, x = Sy),$$

separation $0 = S0 \to \bot$, and contextual induction restricted to equations

$$0 = t(n, \vec{p})$$

(or, equivalently, $t(n, \vec{p}) = 0$). Call this theory T_0. Adding successor injectivity will not affect the counterexample.

A tempting simplification is to use progressive introduction only when a matrix equation has literal zero on one side, and to use the case introduction otherwise. The following theorem rules out the intended recovery of the existing full equation-induction rule from this restriction.

Theorem 11.1 (A model separating zero induction from equation induction)**.** *There is a classical model of T_0, also satisfying successor injectivity, in which*

$$\forall n\, \forall a\; I(n, a; u.u) = a$$

is false. Nevertheless its zero instance and its ordinary induction step are theorems of the computation equations and equality logic. Consequently induction on arbitrary literal equations is not admissible in T_0.

The model is a classical one-world model and hence also a model for intuitionistic first-order logic. Its construction takes place in ordinary classical mathematics, using compactness and choices of compatible paths. These are metatheoretic resources for constructing a countermodel, not extra principles granted to T_0.

11.1 An arithmetic base with two-point fibers

Fix a nonstandard model M of full Peano arithmetic expanded by the usual symbols for all fixed primitive-recursive descriptions. Such a model exists in ordinary mathematics: add a constant larger than each standard numeral, use the standard natural-number model for every finite subtheory, and apply first-order compactness. A countable model may be chosen. We use its full induction only in the external countermodel construction.

For $m \in M$, define the finite nonempty set

$$A_m = \begin{cases} \{0\}, & m \text{ is a standard numeral,} \\ \{0,1\}, & m \text{ is nonstandard.} \end{cases}$$

The distinction is external; it need not be definable in M. Let

$$D := \{(m,i) : m \in M,\ i \in A_m\}, \qquad \pi(m,i) := m.$$

Define

$$0_D = (0_M, 0), \qquad S_D(m,i) = (S_M m, i).$$

This is well defined because successor preserves standardness and nonstandardness.

The map π is surjective and preserves zero and successor. Every standard numeral has a singleton fiber; every nonstandard element has two lifts. Successor is injective on D, and zero is outside its image. Coverage holds: a positive standard numeral has its usual predecessor, and a nonstandard (m,i) has predecessor $(m-1,i)$.

The nonstandard part of M, under successor and predecessor, is a disjoint union of externally bi-infinite chains. More precisely, each equivalence class under a difference of a standard integer is a copy of $\mathbb{Z}$. On D, the label i is retained along each such counter chain.

11.2 Compatible paths through finite fibers

Lemma 11.2 (Finite-fiber path lemma)**.** *Let $(B_j)_{j\in\mathbb{Z}}$ be finite nonempty sets and let $f_j : B_j \to B_{j+1}$ be total maps. There is a sequence $(b_j)_{j\in\mathbb{Z}}$ such that*

$$b_j \in B_j, \qquad b_{j+1} = f_j(b_j)$$

for every j.

Proof. The product $\prod_{j\in\mathbb{Z}} B_j$, with the finite discrete factors, is compact. Each adjacent-coordinate equation is a closed constraint. Every finite collection of these constraints is satisfiable: choose a value at the least involved index and propagate it forward, choosing arbitrary values elsewhere. The finite-intersection property therefore gives a simultaneous solution. Equivalently, this is the usual compactness argument for paths in a sequence of finite nonempty layers. Surjectivity of the transition maps is not required. □

11.3 Choosing iterators by actual body functions

It is essential not to make independent choices merely for syntactic descriptions of bodies. Two bodies that denote the same actual function must receive the same iterator. The following construction ensures this and makes ordinary substitution under the binder legitimate.

Let $\mathscr{H}$ be the set of all actual functions $h : D \to D$ for which there is a unary primitive-recursive function with parameters $f : M \to M$ satisfying

$$\pi(h(x)) = f(\pi(x)) \qquad (x \in D). \tag{Projection}$$

Surjectivity of π makes f unique as an actual function on M.

For such an f, write $\mathrm{It}_f(m,b)$ for its usual iteration in M, with initial value b. This notation does not depend on the chosen primitive-recursive description and parameter tuple for f. Indeed, if two descriptions give the same function on all of M, their two iterators have the same base value and

pointwise identical updates; the equation-induction instance in M comparing them proves equality at every counter. This is a use of induction in the external base model, not in the weak theory T_0.

We define one external operation

$$\mathcal{I} : D \times D \times \mathscr{H} \longrightarrow D.$$

It will obey the scalar computation equations and satisfy

$$\pi(\mathcal{I}(n, a, h)) = \mathrm{It}_f(\pi(n), \pi(a)). \qquad \text{(Iterator projection)}$$

If n is a standard counter k, put

$$\mathcal{I}(k, a, h) := h^k(a),$$

where the right side is actual finite iteration on D.

Now fix a, h and one nonstandard counter chain

$$n_j = (m_j, i), \qquad m_{j+1} = S_M m_j, \qquad j \in \mathbb{Z}.$$

Its counter label i is fixed. Put

$$r_j := \mathrm{It}_f(m_j, \pi(a)), \qquad B_j := \pi^{-1}(r_j).$$

The sets B_j have one or two elements. By the computation law in M, $r_{j+1} = f(r_j)$. Equation (Projection) therefore makes the restriction of h a total map

$$h : B_j \longrightarrow B_{j+1}.$$

Lemma 11.2 gives a compatible path $b_j \in B_j$. Choose one such path and define

$$\mathcal{I}(n_j, a, h) := b_j.$$

Do this for every triple consisting of an actual function h, an initializer a, and a nonstandard counter chain. The choices may be made simultaneously in the metatheory.

There is one specified family of choices. If $h = \mathrm{id}_D$, choose on every nonstandard counter chain the constant path

$$\mathcal{I}(n, a, \mathrm{id}_D) := (\pi(a), 0). \qquad \text{(Identity choice)}$$

This is legitimate: the projected iterator is constantly $\pi(a)$, the label-zero point always exists, and the identity update fixes it. These choices depend on the actual function h, so every syntactic presentation of the identity body uses the same family.

By construction,

$$\mathcal{I}(0_D, a, h) = a, \qquad \mathcal{I}(S_D n, a, h) = h(\mathcal{I}(n, a, h))$$

for every n, a, h, and (Iterator projection) holds. The standard and nonstandard counter components never meet under a finite successor step, so their definitions are compatible.

11.4 Interpreting every finite scalar term

Interpret the binding constructor by

$$[\![I(n, a; u.h)]\!]_D = \mathcal{I}([\![n]\!]_D, [\![a]\!]_D, v \mapsto [\![h]\!]_{D,u:=v}).$$

Lemma 11.3 (Term projection and absence of syntactic circularity)**.** *Every finite scalar term has a well-defined interpretation on D. For every valuation ρ,*

$$\pi([\![t]\!]_{D,\rho}) = [\![t]\!]_{M,\pi\circ\rho},$$

where the right side uses the ordinary initialized scalar iterator in M. In particular every body function supplied to $\mathcal{I}$ belongs to $\mathscr{H}$.

Proof. Induct externally on the finite syntax, treating a body's bound variable as an additional free numerical input in the induction hypothesis. Zero, successor, and variables are immediate. The induction hypothesis for a body gives a function satisfying (Projection), whose projected function is the fixed primitive-recursive interpretation of that body, with its fixed parameter values in M. Thus it lies in $\mathscr{H}$, and (Iterator projection) proves the assertion for the enclosing iterator.

The operation $\mathcal{I}$ was chosen for actual functions in $\mathscr{H}$ before this syntax induction; its definition does not call an evaluator for the term language. Finite nesting therefore creates no circular semantic definition. Equivalently, one obtains a first-order structure by taking one function symbol for every finite earlier-stage body and giving its numerical arguments the interpretation displayed above. Equal parameter values determine the same actual body function and hence the same iterator. The usual substitution lemma, including substitution under a recursor body after renaming bound variables, follows from compositional semantics. □

This also shows that the two scalar computation schemas hold for every fixed finite body, including nested descriptions. No uniform total evaluator is postulated in the object language.

11.5 Why every zero-equation induction instance holds

A singleton zero fiber gives

$$[\![t]\!]_D = 0_D \quad \Longleftrightarrow \quad [\![t]\!]_M = 0_M$$

after projecting the valuation.

Fix numerical parameters $\vec{a} \in D$ and a scalar term $t(n, \vec{a})$. Suppose in D that

$$t(0, \vec{a}) = 0, \qquad \forall n \, (t(n, \vec{a}) = 0 \to t(Sn, \vec{a}) = 0).$$

Write $\bar{t}$ for the primitive-recursive interpretation in M. Term projection and surjectivity of π imply

$$\bar{t}(0, \pi\vec{a}) = 0, \qquad \forall m \in M \, (\bar{t}(m, \pi\vec{a}) = 0 \to \bar{t}(Sm, \pi\vec{a}) = 0).$$

Apply the relevant induction instance in M, then reflect zero back through π, to obtain $\forall n \in D \, t(n, \vec{a}) = 0$.

This validates contextual zero-equation induction as well. At any valuation where the retained context is true, the two premises supply exactly the displayed base and universal step; no projection of the possibly complicated retained context is required. Thus arbitrary first-order side assumptions do not repair the missing induction strength.

In fact the same argument validates induction on every equation

$$\overline{k} = t(n, \vec{a})$$

with one side a fixed standard numeral, because every such numeral has a singleton fiber. Allowing every fixed numeral in place of zero still does not recover full equation induction.

11.6 Failure of identity-iterator uniqueness

Choose a nonstandard $b \in M$ and let $a = (b, 1)$. For the identity body define

$$E(n): \quad I(n, a; u.u) = a.$$

Its base is a computation equation. Its step is provable using

$$I(Sn, a; u.u) = I(n, a; u.u):$$

under $E(n)$, equality transitivity gives $E(Sn)$. Hence these are genuinely provable induction premises, not merely premises accidentally true in the constructed model.

For every nonstandard counter n, however, (Identity choice) gives

$$I(n, a; u.u) = (b, 0) \neq (b, 1) = a.$$

This proves Theorem 11.1. The failed equality is between two elements with the same projection; zero tests cannot distinguish the two labels.

Corollary 11.4 (No recovery of actual-equality reflection)**.** *The theory T_0 cannot prove, for a scalar term $e(x, y)$, the reflection law*

$$e(x, y) = 0 \quad \longleftrightarrow \quad x = y.$$

Proof. Take the distinct points $x = (b, 0)$, $y = (b, 1)$ over one nonstandard b. Term projection gives the same projected value to $e(x, y)$ and $e(x, x)$. If the proposed reflection law held, $e(x, x) = 0$; the singleton zero fiber would then force $e(x, y) = 0$ also, contradicting $x \neq y$. □

The usual conversion of arbitrary equations into zero equations therefore cannot be used to justify this induction restriction: in this scalar presentation the required actual-equality reflection is precisely a missing conclusion.

Exact scope. The theorem refutes recovery of the current scalar equation-induction rule, even with full intuitionistic first-order equality logic, all numerical cases, both computation schemas, and successor injectivity available. In the native Q, D presentation, the ordinary readings make QE, DE, case introduction, and progress for zero equations valid in this same model; hence those logical rules do not evade the counterexample.

The construction does not prove that T_0 has no interpretation of arithmetic on a quotient, and does not prove that every possible changed-domain bi-interpretation with iPRA is impossible. Such claims require additional arguments. Its precise conclusion is that the proposed literal-zero progress restriction does not preserve the existing same-domain, actual-equality consequence relation.

Further research suggested by the countermodel. A useful remaining question is to characterize which additional zero-induction principles force the fibers invisible to zero tests to collapse. Actual-equality reflection suffices but is a substantial additional resource. Another candidate is a narrowly stated iterator-uniqueness or extensionality principle that excludes the identity-iterator defect above; its sufficiency for all equation induction, and its complete-checker cost, remain to be proved.

12 Further research questions

1. Can a canonical-R encoding absorb zero-counter erasure into an already necessary traversal and improve the current compact source minima, while validating its complete public language? The normalization must be applied after substitution, not merely when reading the original input.

2. Can the present normalization equivalence be formalized by extending the existing Lean term and native-calculus development? Body extensionality should be proved before normalization soundness; literal-source and witness tests should be handled after unrestricted equivalence.

3. Which additional principle, beyond zero-equation induction, rules out the two-point fibers of Section 11? Actual-equality reflection suffices, but its cost must be counted. Does a narrowly stated identity-iterator uniqueness principle already suffice for full equation induction?

4. Can the new unary obstruction be strengthened to a useful normal form or effective classification of the scalar self-seeded term functions? The proof above supplies existence of a bound, not an algorithm extracting an optimal bound from a term.

5. Can a pointfree proof representation reduce the substitution machinery without introducing arbitrary extra pairing structure? External tuple contexts avoid that interpretation-composite problem, but their typing and conversion costs must be counted.

Research status. The six-rule theorem, unary obstruction, and zero-induction countermodel are written mathematical arguments. The executable checks test the stated implementations, scoping, normalization, and selected derivations; they do not constitute a formal verification of either universal metatheorem. The upstream formalized scalar bridge is used with its stated scope. No global minimality or literature-wide priority theorem is asserted.

Attribution for the inherited appendices

The next seven source sections are reproduced from the pinned ProveIt snapshot under its MIT No Attribution license. They are reference material, not new contributions of this report. Their original notations and internal attribution are retained. The two successor-seeded sections justify the raw R arithmetic used above. The remaining sections give the complete scalar logic and arithmetic bridge. Their separate files are also in the archive. The local files `successor_seed_prelogic.tex` and `successor_seed_bootstrap.tex` are exact copies of `Research/successor_seed_completion/theory/prelogic.tex` and `Research/successor_seed_completion/theory/bootstrap.tex`, respectively.

A Native bootstrap: cases before a full selector

This section checks a dependency that matters for the new scalar basis. Before predecessor is available, the partial selector

$$B(n,a,b) = J(n,a;u.b), \qquad B(0,a,b) = Sa, \quad B(Sn,a,b) = b$$

is not the unrestricted selector $C(0,a,b) = a$, $C(Sn,a,b) = b$. Accordingly, one must not silently invoke the later ordinary-connective macros during the arithmetic bootstrap. In particular, the theorem rule for numerical cases is not by itself a rule for eliminating equations $t = 0$ and $t = Sz$ from a retained context. The finite-case lemma below supplies exactly the stronger form used by the bootstrap.

We use the native Eq/Q/D presentation with hypothesis use, full formula transport, Q elimination, paired elimination, and the two paired introductions. In notation,

$$Q_y F = \text{the clause taking a proof of } F(0) \text{ to } F(t),$$

$$\frac{\Gamma, F(t,0) \vdash F(t,Sz)}{\Gamma \vdash D_{x,y}F(x,y)}, \qquad \frac{\Gamma, E(t,z) \vdash E(t,Sz)}{\Gamma \vdash D_{x,y}E(x,y)}.$$

The second matrix is a literal equation. The usual freshness conditions are retained, and paired elimination opens $Q_y F(a,y)$ at a fresh a. Computation equations may be read as leaves, or obtained from their constant generated-assumption instances and hypothesis use.

A.1 Resources available without a numerical selector

Choose v fresh for t and put $r = J(S0,0;v.t)$. Computation gives $r = t$. Transport this equation along itself with motive $w = t$ to obtain $t = t$. The familiar equality motives then give symmetry, transitivity and numerical congruence; this uses neither predecessor nor successor injectivity.

For any native formula template F, case introduction with an unused witness coordinate gives $D_{x,y}F(y)$ from $\Gamma \vdash F(Sz)$, ignoring the retained $F(0)$. Paired elimination opens $Q_y F(y)$; a supplied proof of $F(0)$ followed by Q elimination at t gives

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}. \tag{Native NC}$$

The identical construction with progress introduction, using its local $E(z)$, gives contextual literal-equation induction from a base and step. These arguments use no arithmetic selector or ordinary logical connective.

For reference, $Q_y F$ and $D_{x,y}F$, with x unused, are interderivable already: use case introduction and Q elimination in the forward direction, and paired elimination followed by hypothesis use in

the reverse direction. Thus the additional Q constructor presents the same internal clauses as the Eq/D formulation.

Even equation explosion is available at this stage. Under $0 = 1$, congruence gives $B(0, 0, t) = B(1, 0, t)$; computation gives $1 = t$. Consequently every equation follows. External induction on fixed native formula syntax, using case introduction (and the unused-witness representation for Q), gives explosion for every native formula. This is an external proof transformation, not compound-formula induction inside the arithmetic theory.

A.2 Functional finite ranges do not assume branch equations

Say that a particular term b has the *functional range rule* $\mathrm{FR}_R(b)$, for a finite list of numerals R, when the following metarule is derived for every native formula template K:

$$(\Gamma \vdash K(r) \text{ for each } r \in R) \quad \Longrightarrow \quad \Gamma \vdash K(b). \tag{FR}$$

There are no added equations $b = r$ in these premises. The retained context and every fixed parameter of K remain unchanged. In particular, they may mention the original term b.

Use the fixed terms Z, N, j, M defined in Section B.3, with $Z(t) = B(t, 0, 0)$. The present proof checks their logical prerequisites; their finite computation identities are verified directly from those definitions.

For $Z(t)$, apply Native NC to the fresh template $w \mapsto K(Z(w))$. The zero branch is $K(1)$ and every successor branch is $K(0)$, by computation and transport. Hence $\mathrm{FR}_{\{0,1\}}(Z(t))$ holds, and therefore so does the same rule for $N(t)$. For $B(t, 1, 0)$ the identical construction has endpoints two and zero. For

$$j(t) = B(t, 0, B(Z(N(t)), 1, 0))$$

use NC on the outer control: the zero branch is one, and the successor branch has the preceding two-value rule. This proves $\mathrm{FR}_{\{0,1,2\}}(j(t))$, and hence the same rule for $M(t)$, since M is an outer application of j.

The inner parity iterate $U(t) = J(t, 0; u.Z(u))$ has a two-value range rule as well. At counter zero it is one; at a successor it is a value of Z. Apply NC on the counter and the already proved range rule of Z. No induction with a disjunctive invariant is involved.

These constructions justify the cycle equations before contextual finite-case elimination is used. For example, $Z(Z(U(t))) = U(t)$ follows by applying its two-value rule to the equation template $Z(Z(w)) = w$. The equations

$$M(0) = 0, \quad M(1) = 1, \quad M(2) = 2, \qquad M(Sw) = j(M(w))$$

and $j(0) = 1$, $j(1) = 2$, $j(2) = 0$ then follow from the fixed definitions. Their functional range proofs are independent of the ramp invariant.

Lemma A.1 (Successor cancellation on the three finite values)**.** *Suppose b and c have functional range rules whose values are contained in $\{0, 1, 2\}$. In any retained context,*

$$Sb = Sc \quad \Longrightarrow \quad b = c$$

is derivable, before global successor injectivity is known.

Proof. Apply the two functional range rules to the equation templates $M(w) = w$ and $j(j(j(w))) = w$. Their required numeral instances are fixed computations. Thus $M(b) = b$, $M(c) = c$, $j^3(b) = b$ and $j^3(c) = c$ are already theorems in the retained context. Apply M to $Sb = Sc$ and use

$M(Sw) = j(M(w))$ to get $j(b) = j(c)$. Apply j twice and use the two cycle identities to obtain $b = c$. Every step is ordinary numerical congruence or transport. In particular, no cancellation of an arbitrary successor was inserted. □

Lemma A.2 (Contextual finite-case elimination for equations)**.** *Let b have $\mathrm{FR}_R(b)$, where R is contained in $\{0, 1, 2\}$, and let F be the literal equation $u = v$. If, for each $r \in R$, there is a derivation*

$$\Gamma, b = r \vdash u = v,$$

then $\Gamma \vdash u = v$ is derivable. The context may contain arbitrary native formulas, including an induction hypothesis that depends on b.

Proof. Define a native formula template, with both bound variables fresh,

$$K(x) := D_{a,y}(B(y, x, u) = B(y, b, v)),$$

where a is unused. Fix $r \in R$. To introduce $K(r)$ by the case choice with witness zero, its local zero assumption reduces by computation to $Sr = Sb$. Lemma A.1, with the constant r and the known finite-range term b, yields $b = r$. Cut in the given branch derivation. Its conclusion $u = v$ is exactly the successor matrix, up to the computations of B. Thus $\Gamma \vdash K(r)$.

Apply the functional range rule of b to these closed branch proofs. It gives $\Gamma \vdash K(b)$, whose matrix is $B(y, b, u) = B(y, b, v)$. Paired elimination opens its Q clause. At zero its two sides are Sb, so reflexivity and computation supply the base. Specializing the clause at one gives $u = v$. Close paired elimination. All temporary coordinates are fresh and all temporary assumptions have been discharged. □

The derivation is a finite guard construction, not a primitive implication connective. It relies only on the partial selector, computations, transport, hypotheses, case introduction, paired elimination and Q elimination. Neither predecessor nor the unrestricted selector occurs.

A.3 Audit of the bootstrap and completion of the bridge

The dependency order is now explicit. First establish functional range rules and the two finite cycles using only Native NC and equations. Then establish finite successor cancellation and Lemma A.2. The finite splits in the ramp induction, the affine scaling induction and the factor-map calculation are instances of that lemma, with $b = N(n)$ or another displayed finite-range term and with a literal-equation conclusion. Retained induction hypotheses are therefore legitimate. The saturation identity $K(c, Su) = SK(c, u)$ is proved directly by NC with no branch assumptions; it does not require an unrestricted contextual split on arbitrary c. No step in this order uses the later ordinary logical macros.

After the corrected predecessor P is constructed, the initializer compiler gives the ordinary scalar I. Consequently

$$C(n, a, b) := I(n, a; u.b)$$

has the unrestricted selector endpoints. The inherited gate, padding, formula-selector and ordinary-connective constructions may now be applied in their stated order. Binder congruence itself uses only case introduction, paired elimination, transport and cut; it is already valid before this final stage.

For clarity, the ordinary interpretations of the native binders are

$$(Q_y B)^\circ = B^\circ(0) \to \forall y\, B^\circ(y), \qquad (D_{x,y} B)^\circ = \exists x (B^\circ(x, 0) \to \forall y\, B^\circ(x, y)).$$

Once the logical macros are available, the native return translations are proved by the same explicit constructions:

$$Q_y B \dashv\vdash B(0) \Rightarrow_* \forall_* y\, B(y),$$
$$D_{x,y} B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big).$$

Forwards, an opened Q clause under its base supplies the arbitrary instance needed by universal introduction. Backwards, a displayed implication and universal supply the successor matrix for case introduction; an unused witness gives the Q case. The existential witness is retained in the D case. External structural induction on finite formulas, together with the endpoint proofs for the ordinary macros, yields both formula composites. Falsity uses the explicit ordinary separation bridge $(0 = 1) \to \bot$.

Finally, the inherited scalar completion and fixed-description PR compiler are used only after the logical interface and initialized I have been recovered. Recurrence uniqueness gives the numerical composites on the whole original domain. The domain is unchanged, interpreted equality is actual equality, and the numerical isomorphism graph is $x = y$. Thus the finite-case repair closes the pre-selector logical dependency; it does not replace the separately supplied complete arithmetic and logical interpretations by standard-model calculations.

B A successor-seeded scalar basis with two computation schemas

This section completes the explicitly unresolved one-variable successor-seeded candidate in the supplied shared-continuation report [2], in its section "A genuinely smaller numerical grammar: the precise unresolved step." The candidate basis and its conditional initializer compiler are inherited from that report. Unlike the counter-aware successor recursor, its update has just one changing input. The initializer remains a separate argument, but the iterator starts at its successor. The missing result is a numerical predecessor term with a whole-domain proof, obtained here by adapting the supplied scalar 2/3 factor-map construction to shifted arithmetic and correcting a saturated predecessor at three small arguments.

B.1 Syntax, computations, and the precise resources used

The terms are finite staged expressions

$$t ::= x \mid 0 \mid J(n, a; u.h).$$

Only h binds u; its other free variables are passive parameters. Define the abbreviation

$$Sa := J(0, a; u.0), \qquad 1 := S0,$$

and use standard numerals as finite strings of this abbreviation. There are exactly two numerical computation schemas:

$$J(0, a; u.h) = Sa, \tag{J0}$$
$$J(Sn, a; u.h) = h[J(n, a; u.h)/u]. \tag{JS}$$

The first schema, with the abbreviation expanded, says that $J(0, a; u.h) = J(0, a; u.0)$. It does not presuppose a separate successor constructor. No function variables or uniform evaluator are present.

The bootstrap uses equality replacement, numerical cases

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)}, \tag{NC}$$

and contextual induction only on a literal equation E:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}. \tag{EI}$$

The ordinary eigenvariable conditions apply. In an ordinary intuitionistic first-order presentation, NC follows from coverage $\forall x\,(x = 0 \vee \exists y\, x = Sy)$. In the native paired-binder presentation it is the already available case rule. No use of successor injectivity, predecessor, pairing, an equality characteristic, or induction on a compound formula is made before predecessor is constructed below.

The finite range splits used below require the distinction proved in Section A. Functional range elimination follows from NC; contextual finite cases with retained induction hypotheses use the finite cancellation and guard construction of Lemma A.2. The cycle identities are established first by functional range elimination alone. Thus a displayed finite range is not an uncharged disjunction or an assumption of the ordinary-connective macros.

We shall use the following admissible consequence of EI. If two fixed updates are pointwise equal, and the displayed initial values agree, their iterates agree at every counter. Induct on the literal equation between the two iterates: J0 supplies its base and JS supplies its step. This is a derived uniqueness argument, not a primitive extensionality rule for update bodies. All passive arguments are kept fixed while performing the induction.

B.2 A partial selector and shifted addition

Put

$$B(n,a,b) := J(n,a;u.b), \qquad Z(n) := B(n,0,0), \qquad a \oplus b := J(b,a;u.Su).$$

Their computation endpoints are

$$B(0,a,b) = Sa, \quad B(Sn,a,b) = b, \qquad Z(0) = 1, \quad Z(Sn) = 0,$$

$$a \oplus 0 = Sa, \qquad a \oplus Sb = S(a \oplus b).$$

The operation $\oplus$ has standard meaning $a + b + 1$. Ordinary unshifted addition has not been defined and is not used in the proof. Literal-equation induction proves

$$0 \oplus a = Sa, \quad Sa \oplus b = S(a \oplus b), \quad a \oplus b = b \oplus a, \quad (a \oplus b) \oplus c = a \oplus (b \oplus c). \tag{Shifted algebra}$$

For the first two equations induct on the second argument. For commutativity induct on b, using the first two equations. For associativity induct on c; at zero both sides are $S(a \oplus b)$, and the successor step applies S to the previous equality. These proofs do not cancel a successor.

In particular, the following are genuine available terms:

$$D(a) := S^2(a \oplus a), \qquad T(a) := S^4((a \oplus a) \oplus a), \qquad R(a) := S^3(a \oplus a). \tag{Affine terms}$$

Their standard meanings are respectively $2a + 3$, $3a + 6$, and $2a + 4$. The proof uses their displayed term definitions. From (Shifted algebra) one obtains

$$D(0) = 3, \quad D(Sa) = S^2 D(a), \qquad T(0) = 6, \quad T(Sa) = S^3 T(a). \tag{Affine shifts}$$

B.3 Two finite cycles

Define

$$N(n) := Z\big(J(n, 0; u.Z(u))\big).$$

The inner iterator is always zero or one: at zero it is one, and at a successor it is a value of Z. The endpoints of Z and finite cases therefore give

$$N(0) = 0, \quad N(Sn) = Z(N(n)), \quad N(S^2n) = N(n), \qquad N(n) \in \{0, 1\}. \tag{Parity cycle}$$

This asserts a proved two-state recurrence, not a pre-existing parity operation. For example, if $K(n) = J(n, 0; u.Z(u))$, then $N(Sn) = Z(Z(K(n))) = K(n) = Z(N(n))$ by its two cases.

Next put

$$j(u) := B\big(u, 0, B(Z(N(u)), 1, 0)\big), \qquad M(n) := j\big(j(J(n, 0; u.j(u)))\big).$$

Finite computations give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The range of j is contained in $\{0, 1, 2\}$: its zero branch is one, and its successor branch is zero or two according to $N(u)$. Consequently the definitions and these three computations give

$$M(0) = 0, \quad M(Sn) = j(M(n)), \quad M(S^3n) = M(n), \qquad M(n) \in \{0, 1, 2\}. \tag{Three-cycle}$$

Here $M(0) = j(j(1)) = 0$, and the successor equation follows by associativity of composing the fixed term j with itself. The period three follows from its three possible values. No numerical division algorithm has been assumed.

B.4 The shifted ramp and factor map

Set

$$G(n) := J(n, 0; u.u \oplus M(S^2u)).$$

Its first values are

$$G(0) = 1,\ G(1) = 2,\ G(2) = 4,\ G(3) = 5,\ G(4) = 7,\ G(5) = 8,\ G(6) = 10.$$

Lemma B.1 (Ramp identities)**.** *The preceding theory proves*

$$M(G(n)) = S(N(n)), \qquad G(S^2n) = S^3G(n), \tag{Ramp}$$

and admits the following two finite-case calculations:

	$N(n) = 0$	$N(n) = 1$
$G(Sn)$	$SG(n)$	$S^2G(n)$
$G(S^3n)$	$S^4G(n)$	$S^5G(n)$.

(Ramp cases)

Proof. Induct on the single equation $M(G(n)) = S(N(n))$. Its base is $M(1) = 1$. If $N(n) = 0$, the induction equation gives $M(G(n)) = 1$, hence $M(S^2G(n)) = 0$ by the three-cycle. The next state is $G(n) \oplus 0 = SG(n)$, whose M-value is two, as required. If $N(n) = 1$, the current M-value is two and $M(S^2G(n)) = 1$. The next state is $G(n) \oplus 1 = S^2G(n)$, whose M-value is one. This proves the invariant and the first line of the case table. Applying that line twice, with the parity cycle, gives the two-step equation; applying it three times gives the last line. □

Lemma B.2 (Affine identities)**.** *Throughout the entire numerical domain,*

$$N(D(a)) = 1, \qquad N(T(a)) = N(a), \qquad SG(D(a)) = T(a), \qquad \text{(Affine parity and ramp)}$$

$$G(T(a)) = T(G(a)) \oplus N(a). \qquad \text{(Ramp scaling)}$$

Proof. For $N(D(a)) = 1$, use EI, $D(0) = 3$, and the two-step parity period. For $N(T(a)) = N(a)$ use EI, $T(0) = 6$, and $T(Sa) = S^3T(a)$; three parity steps toggle both sides once.

For $SG(D(a)) = T(a)$ use EI on that literal equation. The base is $SG(3) = 6$. The successor step uses $D(Sa) = S^2D(a)$ and $G(S^2v) = S^3G(v)$, so it applies three successors to the previous equation, just as T does.

For (Ramp scaling), the base is $G(6) = 10 = T(1) \oplus 0$. Retain its equation as the induction hypothesis, and split $N(a)$. In the zero case, $N(T(a)) = 0$, so the new left side is $S^4G(T(a))$. The old right side is $T(G(a)) \oplus 0 = ST(G(a))$, while the new right side is

$$T(SG(a)) \oplus 1 = S^5T(G(a)).$$

Thus both sides have changed by four successors. In the one case, the new left side is $S^5G(T(a))$. The old right side is $T(G(a)) \oplus 1 = S^2T(G(a))$, while the new right side is

$$T(S^2G(a)) \oplus 0 = S^7T(G(a)).$$

Thus both sides have changed by five successors. These calculations use only (Shifted algebra), (Affine shifts), and the ramp case table; no successor has been cancelled. □

Define the factor map

$$H(u) := B(Z(N(u)), G(u), u).$$

Its two cases are

$$N(u) = 0 \implies H(u) = u, \qquad N(u) = 1 \implies H(u) = SG(u).$$

Why these shifts are useful. For intuition on standard natural numbers only, $G(u) = \lfloor 3u/2 \rfloor + 1$. Thus H fixes even inputs and sends an odd u to $3(u+1)/2$. After shifting the state by three, an active step multiplies it by $3/2$. The affine maps $D(a) = 2a + 3$ and $T(a) = 3a + 6$ are precisely doubling and tripling in this shifted coordinate: $D(a) + 3 = 2(a + 3)$ and $T(a) + 3 = 3(a + 3)$. Starting at two makes the unshifted base even and fixed by H, while its shifted value is five. The later seed sequence therefore has standard values $5 \cdot 2^x - 3$. This motivates a comparator that tests when its powers of two have been exhausted. The proof below uses the displayed J terms and literal equations; division, exponentiation, and this standard-value description are not prerequisites.

Lemma B.3 (The factor-map equations)**.** *The following global equations hold:*

$$H(2) = 2, \qquad H(D(a)) = T(a), \qquad H(T(a)) = T(H(a)). \qquad \text{(Factor)}$$

Proof. The first equation follows from $N(2) = 0$. Since $N(D(a)) = 1$, the second is $SG(D(a)) = T(a)$. For the last equation split $N(a)$, noting that $N(T(a)) = N(a)$. In the zero case both sides equal $T(a)$. In the one case, (Ramp scaling) gives

$$H(T(a)) = SG(T(a)) = S(T(G(a)) \oplus 1) = S^3T(G(a)) = T(SG(a)) = T(H(a)).$$

□

B.5 A comparator without stripping an initializer

Put

$$A(x) := J(x, 0; u.R(u)), \qquad E(x) := SA(x),$$
$$F(x, y) := J(y, A(x); u.H(u)), \qquad L(x, y) := Z(N(F(x, y))).$$

By direct computation and shifted algebra,

$$E(0) = 2, \qquad E(Sx) = D(E(x)), \qquad F(x, 0) = E(x).$$

The definition of F is the decisive initializer alignment: its successor-seeded initial value is exactly $E(x) = SA(x)$. There is no hidden use of predecessor to initialize it at $E(x)$.

Lemma B.4 (Comparator equations)**.** *The term L is zero or one and satisfies*

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{Comparator}$$

Proof. Its range follows from the range of Z. EI on y proves $F(0, y) = 2$, since $E(0) = 2$ and $H(2) = 2$; hence $L(0, y) = 1$. Also $F(Sx, 0) = E(Sx) = D(E(x))$ has parity one, giving $L(Sx, 0) = 0$.

For the remaining equation first prove, by EI on y with x fixed,

$$F(Sx, Sy) = T(F(x, y)).$$

At zero, the left side is $H(E(Sx)) = H(D(E(x))) = T(E(x))$. In the successor step both iterates apply H, and $H \circ T = T \circ H$ rewrites the result into $T(F(x, Sy))$. Applying $Z \circ N$ and using $N(T(v)) = N(v)$ gives the desired comparator equation.

This proof avoids cancellation in an equation between two successor initializers. Such cancellation would be unavailable at this stage. □

B.6 Saturation and a finite correction give true predecessor

Define conditional increment by

$$K(c, u) := B(c, u, u), \qquad B_m(u) := K(Z(L(Su, m)), u), \qquad V(t, m) := J(t, 0; u.B_m(u)).$$

Thus $K(0, u) = Su$ and $K(Sv, u) = u$. The counter m is a passive parameter of the iteration defining V.

Lemma B.5 (Shifted saturation)**.** *The theory proves*

$$V(St, S^2k) = SV(t, Sk), \qquad V(n, Sn) = Sn, \qquad V(Sn, Sn) = V(S^2n, Sn) = Sn. \tag{Saturation}$$

Proof. Numerical cases on c give $K(c, Su) = SK(c, u)$. Consequently the comparator equations give

$$B_{Sm}(Su) = SB_m(u), \qquad B_{S^2k}(1) = 2.$$

EI on t, with k fixed, now proves the first displayed equation. At zero its two sides equal two. At a successor, apply the defining update to the induction equation and then use the conjugacy $B_{S^2k}(Su) = SB_{Sk}(u)$.

EI gives $L(Sn, n) = 0$ directly from the comparator equations. A separate EI proves $V(n, Sn) = Sn$: the base is $V(0, 1) = 1$, and the successor step uses the previously proved uniform shift at $t = k = n$. This is substitution into a completed lemma, not a change of a passive parameter inside an active induction hypothesis. Finally

$$B_{Sn}(Sn) = K(Z(L(S^2n, Sn)), Sn) = K(Z(L(Sn, n)), Sn) = K(1, Sn) = Sn.$$

One and then two more iterator steps give the last two equations. □

Put

$$Q(m) := J(m, 0; u.K(Z(L(S^2u, m)), u)).$$

This term is a capped predecessor rather than the desired predecessor:

$$Q(0) = Q(1) = 1, \qquad Q(S^2k) = Sk. \tag{Capped predecessor}$$

The first two equations are direct computations. For the last,

$$L(S^2u, S^2k) = L(u, k) = L(Su, Sk)$$

identifies its update with $B_{Sk}(u)$. The derived iterator uniqueness lemma, proved at an independent counter t with k fixed, gives

$$J(t, 0; u.K(Z(L(S^2u, S^2k)), u)) = V(t, Sk).$$

Substitute $t = S^2k$ and apply (Saturation).

Now define one fixed correction term

$$\boxed{P(m) := B(Z(L(3, m)),\ Q(Q(m)),\ L(2, m)).} \tag{Predecessor}$$

There is no arbitrary selector hidden in this expression. Its outer operation is the original partial selector B, whose zero branch adds exactly one successor.

Theorem B.6 (Predecessor and initializer recovery)**.** *The two-schema successor-seeded scalar theory proves*

$$P(0) = 0, \qquad P(Sn) = n.$$

For every fixed update h, the finite term

$$I(n, a; v.h) := P(J(n, a; u.S(h[Pu/v]))) \tag{Initializer compiler}$$

satisfies

$$I(0, a; v.h) = a, \qquad I(Sn, a; v.h) = h[I(n, a; v.h)/v].$$

Proof. The comparator equations give the three direct values

$$P(0) = 0, \qquad P(1) = 0, \qquad P(2) = 1.$$

Indeed, at these inputs $L(3, m) = 0$, so the outer B selects $L(2, m)$, whose three values are zero, zero, one. At $m = S^3k$, one has $L(3, m) = 1$. Thus the zero-control branch of B, together with (Capped predecessor), gives

$$P(S^3k) = SQ(Q(S^3k)) = SQ(S^2k) = S^2k.$$

Numerical cases twice on n now give $P(Sn) = n$ from these displayed equations. Alternatively, two literal-equation inductions, first on $P(S^2n) = Sn$ and then on $P(Sn) = n$, give the same conclusion. No earlier cancellation or injectivity was used. Successor injectivity can now be derived by applying P to an equality of successors.

For the initializer compiler, J0 and the proved predecessor equation give the base $P(Sa) = a$. At a successor, JS exposes the term $S(h[P(J(n, a; u.S(h[Pu/v])))/v])$. The outer P removes its displayed successor and leaves exactly $h[I(n, a; v.h)/v]$. There is no invariant about the internal state to prove: the displayed successor in the update is sufficient. □

B.7 Consequences and the bi-interpretation boundary

The construction supplies the full initialized scalar basis $0, S, I$. In particular the unrestricted selector is now the derived term $C(n, a, b) := I(n, a; u.b)$, with its ordinary endpoints a and b. The supplied scalar arithmetic development can therefore be used from this point onward to obtain actual-equality reflection, induction on arbitrary quantifier-free formulas, pairing and projections, and the compiler for every fixed general primitive-recursion description. Those later arguments are not premises of this bootstrap.

In ordinary iPRA, each fixed J description is interpreted by primitive recursion with seed Sa and its displayed scalar update. Both computation schemas, coverage, separation, and literal-equation induction are then provable. Conversely (Initializer compiler), followed by the existing scalar compiler, interprets every fixed iPRA operation. For each fixed description, literal-equation induction proves uniqueness of its defining recurrence after earlier subdescriptions have been identified. Applied to both composite translations, this identifies each numerical operation with its double translation throughout the entire original domain. Both domain formulas are $x = x$ and both numerical isomorphism graphs are $x = y$.

For the ordinary presentations, full intuitionistic logic is preserved. For the native paired-binder presentation, its ordinary logical macros and the formula return-translation proof must still be included; they apply once the derived selector C is available. This establishes bi-interpretability only when that logical bridge and the scalar arithmetic completion accompany the present argument. It does not replace them by agreement merely on the standard natural numbers.

The arithmetic advance is completion of the predecessor obligation left open for this exact candidate in the shared-continuation report [2]: the affine factor-map alignment, capped predecessor, and finite correction (Predecessor) supply its missing proof. The candidate grammar and initializer compiler are not new. Finite cycles, a 2/3 factor map, and saturation are adapted from the supplied ordinary scalar predecessor construction. No global literature priority is asserted.

The now-completed basis has two primitive numerical constructors and two computation schemas, with a single update variable. A smaller complete proof checker remains a separate implementation claim, which must charge the actual decoder, scope checks, logical rules, and driver.

C Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

C.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \quad \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x,y) \qquad \Gamma, \langle B(a,0); y.B(a,y)\rangle \vdash F}{\Gamma \vdash F} \quad \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \quad \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t,0), [B(t,z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y}B(x,y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

C.2 Exact equivalence and the unused-witness construction

Lemma C.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem C.2 (Identity-language equivalence)**.** *For every retained clause context* Γ *and native formula* F*,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t, y)$ and all retained data. Under $\Gamma, E(t, 0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y} E(t, y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t, 0); y.E(t, y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t, 0)$ supplies its antecedent. Use at Sz gives $E(t, Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y} E(x, y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y} E(x, y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t, y)$ and binds only the unused coordinate u.

Proposition C.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of* H *followed by Use. If an old derivation has* N *nodes and* r *atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

C.3 Induction and numerical cases separate cleanly

Lemma C.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma C.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D are derivable without using the progress choice.

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

C.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B), \\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B), \\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z}M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z}G_zM_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z}G_zA(z), \\ \bot_* &:= (0 = 1). \end{aligned} \qquad \text{(Logical macros)}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \qquad \text{(Coverage)}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

C.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x, 0) \to \forall y B^\circ(x, y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x(B(x, 0) \Rightarrow_* \forall_* y\, B(x, y)). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a, 0)$ that clause supplies $B(a, y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem C.2 preserves every native sequent.

C.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

D Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section C recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

D.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.

2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.

3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I\big(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))\big), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem D.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

E Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [3] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

E.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n,a,b) = I(n,a;u.b), \qquad Z(n) = C(n,1,0), \qquad a+b = I(b,a;u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0,a,b) = a, \qquad C(Sn,a,b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0+b=b$ and $Sa+b=S(a+b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a+a$ and $3a$ means $a+a+a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n,0;u.Z(u)), \qquad j(u) = Z(u)+2N(u), \qquad M(n) = I(n,0;u.j(u)).$$

The range of N is contained in $\{0,1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n)+Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n+2) = N(n). \tag{1}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n+1) = 1, \qquad N(3n) = N(n). \tag{2}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

E.2 A ramp with a global scaling identity

Define

$$G(n) = I(n,0;u.u+M(u)+1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{3}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0)=1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n)+2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n+2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{4}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{5}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (5) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{6}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (5); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v+1$.

E.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m+n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{7}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (6), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{8}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{9}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \le y$ is motivation, not a premise of the following argument.

E.4 Saturated iteration gives predecessor

We now need only the three equations (9). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (10) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (11) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{12}$$

The base is (11). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{13}$$

The first uses the comparator recurrence. The second uses (12) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{14}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{15}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (14) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

E.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

F From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [4]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix E, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

F.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{16}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{17}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{18}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{19}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[\big(E(0, \vec{p}) \wedge \forall n \big(E(n, \vec{p}) \longrightarrow E(Sn, \vec{p})\big)\big) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{20}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (20) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

F.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{21}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{22}$$

$$K(0) = 1, \qquad K(Sx) = x. \tag{23}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (16). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (22); symmetry and (19) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{24}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{25}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{26}$$

To prove the range assertion, split (18). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (17) also give, for all u, v,

$$C(u, v, v) = v. \tag{27}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{28}$$

If $u = 1 = S0$, (23) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

F.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{29}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{30}$$

Using these equations and (20), we now derive

$$0 + y = y, \tag{31}$$
$$(Sx) + y = S(x + y), \tag{32}$$
$$d(Sx, Sy) = d(x, y), \tag{33}$$
$$d(Sa, a) = 1. \tag{34}$$

Each induction below has a single equation as its induction formula.

For (31), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (32), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (32) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{35}$$

Indeed, both sides are $S(y + r)$.

For (33), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (34). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (33) and the induction hypothesis.

F.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{36}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (31) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (27).

For the step, assume (36) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (36) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (35) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (36). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{37}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (36). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (34) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{38}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{39}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{40}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (25), which is equivalent to $d(Sa, b) = 1$ by (28), and then to $a = b$ by (37). The range assertion is (24) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (39) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{41}$$

is a theorem, not an extra hypothesis.

F.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s, t), \qquad b_\bot = 1, \tag{42}$$

$$b_{A \wedge D} = C(b_A, b_D, 1), \tag{43}$$

$$b_{A \vee D} = C(b_A, 0, b_D), \tag{44}$$

$$b_{A \to D} = C(b_A, b_D, 0). \tag{45}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$\big(b_A = 0 \vee b_A = 1\big) \quad \text{and} \quad \big(b_A = 0 \longleftrightarrow A\big). \tag{46}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (39)–(40). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (46) for every quantifier-free formula.

F.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (46), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation

induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

G Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [5]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

G.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a, b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n) + 1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a, b) = (a + b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \leq k^2 + a < (k+1)^2$ since $a \leq k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

G.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u, \vec{p}) = \pi(SL(u), h^*(L(u), H(u), \vec{p})),$$

$$V(n, \vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u, \vec{p})), \qquad f^*(n, \vec{p}) = H(V(n, \vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n, \vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

G.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section C give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

H The complete strict canonical checker

The following normalized module is independently executable. Its full cost appears in Section 9.

```
import json
import sys

def fail():
    raise ValueError('invalid certificate')

def rec(n, a, h):
    return [n, a, 0 if n == 0 else h]

def successor(a):
    return [0, a, 0]

def term(t, n):
    if type(t) is int:
        return 0 <= t <= n
    if type(t) is not list:
        return False
    return len(t) == 3 and term(t[0], n) and term(t[1], n) and term(t[2], n + 1) and (t[0] != 0 or
    t[2] == 0)

def formula(a, n):
    if type(a) is not list or not a:
        return False
    if a[0] == '=':
        return len(a) == 3 and term(a[1], n) and term(a[2], n)
    return len(a) == 2 and a[0] in ('Q', 'D') and formula(a[1], n + (1 if a[0] == 'Q' else 2))

def shift_term(t, amount=1, depth=0):
    if type(t) is int:
        return t + amount if t > depth else t
    return rec(shift_term(t[0], amount, depth), shift_term(t[1], amount, depth), shift_term(t[2],
    amount, depth + 1))

def shift_formula(a, amount=1, depth=0):
    if a[0] == '=':
        return ['=', shift_term(a[1], amount, depth), shift_term(a[2], amount, depth)]
    return [a[0], shift_formula(a[1], amount, depth + (1 if a[0] == 'Q' else 2))]

def subst_term(t, images, depth=0):
    if type(t) is int:
        if t <= depth:
            return t
        index = t - depth
        return shift_term(images[index - 1], depth) if index <= len(images) else t - len(images)
    return rec(subst_term(t[0], images, depth), subst_term(t[1], images, depth), subst_term(t[2],
    images, depth + 1))

def subst_formula(a, images, depth=0):
    if a[0] == '=':
        return ['=', subst_term(a[1], images, depth), subst_term(a[2], images, depth)]
    return [a[0], subst_formula(a[1], images, depth + (1 if a[0] == 'Q' else 2))]

def reverse(a):
    return ['=', a[2], a[1]] if a[0] == '=' else a
```

```
def derive(proof, context, scope):
    if type(proof) is not list or not proof or type(proof[0]) is not str:
        fail()
    tag = proof[0]
    if tag == 'h' and len(proof) == 2:
        index = proof[1]
        if type(index) is int and 0 <= index < len(context):
            return context[index]
        fail()
    if tag == 'r' and len(proof) == 4:
        n, a, h = proof[1:]
        if not (term(n, scope) and term(a, scope) and term(h, scope + 1)):
            fail()
        return ['=', rec(successor(n), a, h), subst_term(h, [rec(n, a, h)])]
    if tag == 'x' and len(proof) == 4:
        template, major, minor = proof[1:]
        if not formula(template, scope + 1) or template[0] != '=':
            fail()
        equation = derive(major, context, scope)
        if equation[0] != '=' or equation[1] == 0:
            fail()
        if derive(minor, context, scope) != subst_formula(template, [equation[1]]):
            fail()
        return reverse(subst_formula(template, [equation[2]]))
    if tag == 'q' and len(proof) == 4:
        witness, major, base = proof[1:]
        if not term(witness, scope):
            fail()
        closure = derive(major, context, scope)
        if closure[0] != 'Q':
            fail()
        if derive(base, context, scope) != subst_formula(closure[1], [0]):
            fail()
        return reverse(subst_formula(closure[1], [witness]))
    if tag == 'e' and len(proof) == 4:
        goal, major, branch = proof[1:]
        if not formula(goal, scope):
            fail()
        paired = derive(major, context, scope)
        if paired[0] != 'D':
            fail()
        extended = [shift_formula(a) for a in context] + [['Q', paired[1]]]
        if derive(branch, extended, scope + 1) != shift_formula(goal):
            fail()
        return goal
    if tag == 'i' and len(proof) == 4:
        goal, witness, branch = proof[1:]
        if not formula(goal, scope) or goal[0] != 'D' or (not term(witness, scope)):
            fail()
        body = shift_formula(goal[1], 1, 2)
        lifted_witness = shift_term(witness)
        eta = 1 if goal[1][0] == '=' and witness == 0 else 0
        hypothesis = subst_formula(body, [eta, lifted_witness])
        target = subst_formula(body, [successor(1), lifted_witness])
        extended = [shift_formula(a) for a in context] + [hypothesis]
        if derive(branch, extended, scope + 1) != target:
            fail()
        return goal
```

```
    fail()

def verify(data):
    if type(data) is not dict or set(data) != {'goal', 'proof'}:
        fail()
    if not formula(data['goal'], 0):
        fail()
    if derive(data['proof'], [], 0) != data['goal']:
        fail()
    return True
if __name__ == '__main__':
    try:
        verify(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

I The separate compact scalar byte profile

This implements the existing guarded calculus, with its documented public-input contract. It is not the canonical-R program.

```
import json

def c(a, g=[0], k=0):
    i = lambda a, t=c, d=0: [i(b, t, d + 5 - len(a)) for b in a] if a * 0 != 0 else a if k <= a < d
     else d % 2 and i(t, d=3)
    if i(a) in g:
        return
    m, s, u, t = q()
    a = (m, a) == (s and [s, u] or [t] * 3, i(t, u)) and i(t, s) or a
    h = c(m, g, k) or m * 3 or (u and [[s and [s], u, t], i(t, [s, u, t], 3) if s else u]) or (a :=
     i(*a, t, 2))
    k -= 1
    c(i(a, [k]), g + [i(h, (m or ~len(a) % 2) and k)], k)
q = json.load(open(0)).pop
c(q())
print('ACCEPT')
```

References

[1] Vladimir Reshetnikov and ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*. Snapshot `8eb71c1e570e07ef45ead686628e77f18ef34798`. Pinned repository collection.

[2] ProveIt contributors. Successor-seeded and shared-continuation research lineage, `Research/successor_seed_completion/theory/prelogic.tex` and `Research/successor_seed_completion/theory/bootstrap.tex`, in the snapshot above.

[3] ProveIt contributors. Scalar predecessor bootstrap, `Research/certified_zero/reference/scalar_predecessor.tex`, with its inherited attribution, in the snapshot above.

[4] ProveIt contributors. Equality reflection and quantifier-free induction, `Research/certified_zero/reference/equality_reflection.tex`, with its inherited attribution, in the snapshot above.

[5] ProveIt contributors. Primitive-recursion compiler, `Research/certified_zero/reference/primitive_recursion.tex`, with its inherited progress-closure attribution, in the snapshot above.

[6] ProveIt contributors. Finite-control and relational obstructions, `Research/local_scopes/theory/finite_profiles.tex` and `relational_obstruction.tex`, in the snapshot above.

[7] ProveIt contributors. Native kernel comparisons, `Lean/ConciseIPRA/NativeMinimal.lean` and `NativeReversal.lean`, and the ordinary scalar bridge, `NativeBiInterpretation.lean`, in the snapshot above.